

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]

Lecture #03: Database Storage (Part I)

15-445/645 Database Systems (Fall 2019)

<https://15445.courses.cs.cmu.edu/fall2019/>

Carnegie Mellon University

Prof. Andy Pavlo

1 Storage

We will focus on a “disk-oriented” DBMS architecture that assumes that primary storage location of the database is on non-volatile disk.

At the top of the storage hierarchy you have the devices that are closest to the CPU. This is the fastest storage but it is also the smallest and most expensive. The further you get away from the CPU, the storage devices have larger the capacities but are much slower and farther away from the CPU. These devices also get cheaper per GB.

Volatile Devices:

- Volatile means that if you pull the power from the machine, then the data is lost.
- Volatile storage supports fast random access with byte-addressable locations. This means that the program can jump to any byte address and get the data that is there.
- For our purposes, we will always refer to this storage class as “memory”.

Non-Volatile Devices:

- Non-volatile means that the storage device does not need to be provided continuous power in order for the device to retain the bits that it is storing.
- It is also block/page addressable. This means that in order to read a value at a particular offset, the program first has to load the 4 KB page into memory that holds the value the program wants to read.
- Non-volatile storage is traditionally better at sequential access (reading multiple chunks of data at the same time).
- We will refer to this as “disk”. We will not make a (major) distinction between solid-state storage (SSD) or spinning hard drives (HDD).

There is also a new class of storage devices that are coming out soon called *non-volatile memory*. These devices are designed to be the best of both worlds: almost as fast as DRAM but with the persistence of disk. We will not cover these devices.

Since the system assumes that the database is stored on disk, the components of the DBMS are responsible for figuring out how to move the data back and forth from the non-volatile disk and volatile memory, since the system cannot operate on the data directly on disk.

We will focus on how we can hide the latency of the disk rather than focusing on optimizations with registers and caches, since getting data from disk is so slow. If reading data from the L1 cache reference took half a second, then reading from an SSD would take 1.7 days and reading from an HDD would take 16.5 weeks.

2 Disk-Oriented DBMS Overview

The database is all on disk, and the data in the database files is organized into pages, and the first page is the directory page. In order to operate on the data the DBMS needs to bring the data into memory. It does this

by having a *buffer pool* that manages the movement back and forth between disk and memory. The DBMS also have an execution engine that will execute queries. The execution engine will ask the buffer pool for a specific page, and the buffer pool will take care of bringing that page into memory and giving the execution engine a pointer to the page in memory. The buffer pool manager will ensure that the page is there while the execution engine is operating on that memory.

3 DBMS vs. OS

A high-level design goal of the DBMS is to support databases that exceed the amount of memory available. Since reading/writing to disk is expensive, it must be managed carefully. We do not want large stalls from fetching something from disk to slow down everything else. So we want the DBMS to be able to process other queries while it is waiting to get the data from disk.

This high-level design goal is like virtual memory, where there is a large address space and a place for the OS to bring in pages from disk.

One way to achieve this virtual memory, is by using `mmap` to map the contents of a file in a process address space, which makes the OS responsible for moving pages back and forth between disk and memory. Unfortunately, this means that if `mmap` hits a page fault, this will block the process.

- You never want to use `mmap` in your DBMS if you need to write.
- The DBMS (almost) always wants to control things itself and can do a better job at it since it knows more about the data being accessed and the queries being processed.
- The operating system is not your friend.

It is possible to use the OS by using:

- `madvise`: Tells the OS know when you are planning on reading certain pages.
- `mlock`: Tells the OS to not swap memory ranges out to disk.
- `msync`: Tells the OS to flush memory ranges out to disk.

We do not advise using `mmap` in a DBMS for correctness and performance reasons.

Even though the system will have functionalities that seem like something the OS can provide, Having the DBMS implement these procedures itself gives it better control and performance.

4 File Storage

In its most basic form, a DBMS stores a database as files on disk. Some may use a file hierarchy, others may use a single file (e.g., SQLite).

The OS does not know anything about the contents of these files. Only the DBMS knows how to decipher their contents, since it is encoded in a way specific to the DBMS.

The DBMS's *storage manager* is responsible for managing a database's files. It represents the files as a collection of pages. It also keeps track of what data has been read and written to pages, as well how much free space there is in the pages.

5 Database Pages

The DBMS organizes the database across one or more files in fixed-size blocks of data called *pages*. Pages can contain different kinds of data (tuples, indexes, etc). Most systems will not mix these types within pages. Some systems will require that it is self-contained, meaning that all the information needed to read

each page is on the page itself.

Each page is given a unique identifier. If the database is a single file, then the page id can just be the file offset. Most DBMSs have an indirection layer that maps a page id to a file path and offset. The upper levels of the system will ask for a specific page number and then the storage manager will have to turn that page number into a file and an offset to find the page.

Most DBMSs use fixed-size pages to avoid the engineering overhead needed to support variable-sized pages. For example, with variable-size pages, deleting a page could create a hole in files that the DBMS cannot easily fill with new pages.

There are three concepts of pages in DBMS:

1. Hardware page (usually 4 KB).
2. OS page (4 KB).
3. Database page (1-16 KB).

The storage device guarantees an atomic write of the size of the hardware page. If the hardware page is 4 KB, then when the system tries to write 4 KB to the disk, either all 4 KB will be written, or none of it will. This means that if our database page is larger than our hardware page, the DBMS will have to take extra measures to ensure that the data gets written out safely since the program can get partway through writing a database page to disk when the system crashes.

6 Database Heap

There are a couple ways to find the location of the page a DBMS wants on the disk, and a heap file organization is one of the ways.

A *heap file* is an unordered collection of pages where tuples are stored in random order.

The DBMS can locate a page on disk given a `page_id` by using a linked list of pages or a page directory.

1. **Linked List:** Header page holds pointers to a list of free pages and a list of data pages. However, if the DBMS is looking for a specific page, it has to do a sequential scan on the data page list until it finds the page it is looking for.
2. **Page Directory:** DBMS maintains special pages that track locations of data pages along with the amount of free space on each page.

7 Page Layout

Every page includes a header that records meta-data about the page's contents:

- Page size.
- Checksum.
- DBMS version.
- Transaction visibility.
- Some systems require pages to be self-contained (e.g. Oracle).

A strawman approach to laying out data is to keep track of how many tuples the DBMS has stored in a page and then every time it adds a new tuple, it appends the tuple to the end. However, problems arise when it deletes a tuple or when the tuples have variable-length attributes.

There are two main approaches to laying out data in pages: (1) slotted-pages and (2) log-structured.

Slotted Pages: Page maps slots to offsets.

- Most common approach used in DBMSs today.
- Header keeps track of the number of used slots and the offset of the starting location of last used slot and a slot array, which keeps track of the location of the start of each tuple.
- To add a tuple, the slot array will grow from the beginning to the end, and the data of the tuples will grow from end to the beginning. The page is considered full when the slot array and the tuple data meet.

Log-Structured: Instead of storing tuples, the DBMS only stores log records.

- Stores records to file of how the database was modified (insert, update, deletes).
- To read a record, the DBMS scans the log file backwards and “recreates” the tuple.
- Fast writes, potentially slow reads.
- Works well on append-only storage because the DBMS cannot go back and update the data.
- To avoid long reads the DBMS can have indexes to allow it to jump to specific locations in the log. It can also periodically compact the log (if it had a tuple and then made an update to it, it could compact it down to just inserting the updated tuple). The issue with compaction is the DBMS ends up with write amplification (it re-writes the same data over and over again).

8 Tuple Layout

A tuple is essentially a sequence of bytes. It is DBMS’s job to interpret those bytes into attribute types and values.

Tuple Header: Contains meta-data about the tuple.

- Visibility information for the DBMS’s concurrency control protocol (i.e., information about which transaction created/modified that tuple).
- Bit Map for NULL values.
- Note that the DBMS does not need to store meta-data about the schema of the database here.

Tuple Data: Actual data for attributes.

- Attributes are typically stored in the order that you specify them when you create the table.
- Most DBMSs do not allow a tuple to exceed the size of a page.

Unique Identifier:

- Each tuple in the database is assigned a unique identifier.
- Most common: `page_id + (offset or slot)`.
- An application **cannot** rely on these ids to mean anything.

Denormalized Tuple Data: If two tables are related, the DBMS can “pre-join” them, so the tables end up on the same page. This makes reads faster since the DBMS only has to load in one page rather than two separate pages, but it makes updates more expensive since the DBMS needs more space for each tuple.

CMU · 14-455 | Database System (2019)

CMU 14-455 (2019) · 课程资料包 @ShowMeAI



视频

中英双语字幕



课件

一键打包下载



笔记

官方笔记翻译



代码

作业项目解析



视频 · B 站 [扫码或点击链接]

<https://www.bilibili.com/video/BV1qf4y1J7mX>



课件 & 代码 · 博客 [扫码或点击链接]

<http://blog.showmeai.tech/cmu-14-455/>

高级 SQL

聚合

排序

并发控制

数据库

内存管理

查询规划

分组

树索引

索引构建

查询执行

Awesome AI Courses Notes Cheatsheets 是 [ShowMeAI](#) 资料库的分支系列，覆盖最具知名度的 **TOP50+** 门 AI 课程，旨在为读者和学习者提供一整套高品质中文学习笔记和速查表。

点击课程名称，跳转至课程**资料包**页面，**一键下载**课程全部资料！

机器学习	深度学习	自然语言处理	计算机视觉
Stanford · CS229	Stanford · CS230	Stanford · CS224n	Stanford · CS231n
# Awesome AI Courses Notes Cheatsheets · 持续更新中			
知识图谱	图机器学习	深度强化学习	自动驾驶
Stanford · CS520	Stanford · CS224W	UCBerkeley · CS285	MIT · 6.S094



微信公众号

资料下载方式 2: 扫码点击**底部菜单栏**称为 **AI 内容创作者?** 回复 [添砖加瓦]